DESAFIO BACKEND 2.0

FOCUS TECH

Chat em Tempo Real

Área: Desenvolvimento Backend

Nível: Intermediário

Duração: 3 semanas (21 dias)

1. Sobre o desafio

Desenvolva o backend de uma plataforma de comunicação em tempo real com conversas privadas, grupos, histórico persistente, notificações e indicadores de presença. O sistema deverá permitir que usuários conectados recebam novas mensagens sem atualizar manualmente a aplicação.

2. Objetivo e funcionalidades obrigatórias

- Cadastro e autenticação segura de usuários.
- Conversas privadas entre dois participantes e grupos com controle de membros.
- Envio e recebimento de mensagens em tempo real por WebSocket.
- Persistência, recuperação e paginação do histórico de mensagens.
- Notificações de novas mensagens, inclusive consulta posterior para usuários offline.
- Presença online/offline, considerando múltiplas conexões por usuário.
- Confirmações de leitura e atualizações em tempo real.
- Reconexão e recuperação, pelo histórico, de mensagens não recebidas.
- Comunicação de eventos entre pelo menos duas instâncias backend usando Redis.

3. Liberdade de escolha das tecnologias

Não há linguagem, framework ou biblioteca de aplicação obrigatórios. Cada participante deve definir a stack com base nos requisitos. WebSocket e Redis fazem parte do escopo técnico obrigatório. Deve ser utilizado um banco de dados persistente adequado para as informações do sistema.

Componente Possibilidades (não obrigatórias)

Backend Python/FastAPI; TypeScript/NestJS; Java/Spring Boot; PHP/Laravel; C#/ASP.NET

Banco de dados PostgreSQL, MySQL, MariaDB ou alternativa justificada

Tempo real WebSocket nativo, Socket.IO ou biblioteca compatível

Autenticação JWT, sessão segura ou estratégia equivalente

Testes Pytest, Jest, JUnit, PHPUnit ou equivalentes

A documentação deve justificar as escolhas feitas no processo.

4. Arquitetura e responsabilidades

A arquitetura é livre, desde que preserve separação de responsabilidades. Uma referência possível é:

controllers HTTP e handlers WebSocket para entrada; services para regras de negócio; repositories para persistência; schemas/DTOs para validação; componentes Redis para presença e distribuição de eventos.

HTTP pode atender cadastro, autenticação, listagens, gestão de grupos e histórico. WebSocket deve atender mensagens e eventos instantâneos. Os dois meios podem reutilizar os mesmos serviços de negócio; evite duplicação de autorização e validações.

O participante deve explicar como sua arquitetura permite manutenção, testes e inclusão de funcionalidades, sem necessidade de introduzir complexidade desnecessária.

5. Modelagem de dados

A modelagem é uma decisão do participante. As entidades abaixo são apenas um modelo de referência e podem ser adaptadas desde que as regras sejam preservadas.

Entidade Campos de referência

User id, name, email, passwordHash, createdAt

Conversation id, type, name, createdBy, createdAt

ConversationMember id, conversationId, userId, role, joinedAt

Message id, conversationId, senderId, content, createdAt

MessageRead id, messageId, userId, readAt

Notification id, userId, messageId, isRead, createdAt

Regras mínimas de integridade

- E-mail de usuário único, senha com hash e nunca retornada em respostas.
- Conversas PRIVATE com exatamente dois usuários distintos; GROUP com nome e ao menos um administrador.
- Evitar conversas privadas duplicadas para o mesmo par, inclusive em solicitações simultâneas.

- Participação única e válida por usuário/conversa; membros de grupo podem ter papéis ADMIN e MEMBER.
- Mensagens vinculadas a remetente e conversa; apenas membros autorizados podem enviá-las ou consultá-las.
- Registrar confirmação de leitura sem duplicações indevidas; notificações pertencem a seus destinatários.
- Datas em ISO 8601 UTC e IDs consistentes, com índices e restrições adequados.

6. Contrato HTTP mínimo

Método Rota Ação

POST /api/auth/register Cadastrar

POST /api/auth/login Autenticar

GET /api/users Buscar usuários

POST /api/conversations/private Iniciar conversa privada

GET /api/conversations Listar conversas

GET /api/conversations/:id/messages Consultar histórico paginado

POST /api/groups Criar grupo

POST /api/groups/:id/members Adicionar membro

DELETE /api/groups/:id/members/:userId Remover membro

PATCH /api/messages/:id/read Marcar mensagem como lida

GET /api/notifications Listar notificações

PATCH /api/notifications/:id/read Marcar notificação como lida

Todas as rotas exceto cadastro e login são protegidas. É permitido adaptar nomes ou acrescentar rotas, desde que o contrato utilizado seja claro e documentado.

Exemplo: criação de conversa privada

POST /api/conversations/private

{ "participantId": "user-002" }

Verificar destinatário, proibir conversa consigo mesmo e criar ou recuperar a conversa já existente. A solução deve impedir duplicações em concorrência.

Exemplo: criação de grupo

POST /api/groups

{ "name": "Equipe Backend", "memberIds": ["user-002", "user-003"] }

O criador deve ser administrador; somente administradores podem gerir membros. Validar participantes existentes e impedir duplicatas.

Exemplo: histórico paginado

GET /api/conversations/:id/messages?limit=20&cursor=...

{ "messages": [{ "id": "msg-001", "senderId": "user-001",

"content": "Olá!", "createdAt": "2026-10-16T14:30:00Z" }],

"nextCursor": null }

Exigir participação na conversa, paginação e ordenação determinística. A estratégia de paginação é livre e deve ser explicada.

7. Eventos WebSocket e fluxo de mensagem

Toda conexão WebSocket precisa ser autenticada. O servidor deverá identificar o usuário, validar os eventos e garantir que somente integrantes autorizados recebam informações de cada conversa.

Evento Direção Função

message:send Cliente → servidor Solicitar envio

message:ack Servidor → cliente Confirmar persistência

message:new Servidor → cliente Distribuir mensagem

message:read Cliente → servidor Registrar leitura

message:read:update Servidor → cliente Propagar leitura

notification:new Servidor → cliente Notificar nova mensagem

user:presence Servidor → cliente Atualizar presença

error Servidor → cliente Informar falha estruturada

{ "event": "message:send", "data": { "conversationId": "conv-001",

"clientMessageId": "request-001", "content": "Olá, pessoal!" } }

Processamento obrigatório: autenticar o cliente; conferir participação; validar o conteúdo; persistir a mensagem; confirmar o armazenamento; distribuir aos destinatários conectados; registrar notificações. A confirmação SAVED representa persistência, não garantia de entrega ou leitura.

{ "event": "message:ack", "data": { "clientMessageId": "request-001",

"messageId": "msg-001", "status": "SAVED" } }

O identificador clientMessageId ou mecanismo equivalente deve permitir identificar reenvios. Após desconexão, o cliente deverá conseguir reconectar-se e recuperar

mensagens pelo histórico. Não é exigida entrega garantida de eventos durante a desconexão.

8. Presença, notificações e leitura

ONLINE significa ao menos uma conexão ativa reconhecida; OFFLINE significa nenhuma. O fechamento de apenas uma aba não deve desligar o status de alguém que continua conectado em outro dispositivo. A estratégia deve considerar heartbeat/expiração e quedas inesperadas, quando aplicável.

Novas mensagens devem gerar notificações para destinatários apropriados; usuários conectados devem receber um evento e usuários desconectados devem poder consultar notificações persistidas posteriormente. A marcação de leitura deve identificar leitor e momento, sem permitir que um usuário altere a leitura atribuída a outro.

9. Redis e escalabilidade

Redis é obrigatório para gerenciar informações temporárias de presença e compartilhar eventos entre instâncias por Pub/Sub ou mecanismo Redis justificado que atenda ao mesmo requisito de distribuição. O banco persistente permanece como fonte de verdade das mensagens.

Cenário de demonstração

- Usuário A está conectado ao backend 1 e usuário B ao backend 2.
- A envia uma mensagem; a instância 1 valida e persiste no banco.
- A instância 1 publica evento no Redis; a instância 2 o recebe.
- A instância 2 entrega o evento a B, sem permitir acesso por usuários alheios à conversa.

O participante deve demonstrar duas instâncias conectadas ao mesmo Redis e documentar como evita emissões duplicadas. Redis Pub/Sub não armazena eventos para assinantes desconectados: a recuperação deve ocorrer por leitura do histórico persistente.

10. Concorrência e consistência

O sistema deverá lidar com solicitações simultâneas e situações de falha, preservando os dados e as permissões:

- Dois usuários enviando mensagens para a mesma conversa, sem perda e com ordenação estável no

histórico.

- Cliente reenviando o mesmo pedido após perder a confirmação, sem duplicar uma mensagem.
- Duas solicitações simultâneas de criação da mesma conversa privada, sem duplicação.

- Participante removido de grupo que mantém um WebSocket aberto: deve perder imediatamente a

autorização para novas operações e eventos.

As estratégias adotadas (constraints, transações, idempotência, verificações por evento etc.) são de livre escolha, mas precisam ser documentadas.

11. Validação, erros e segurança

Validar e-mail, senha, IDs, existência e participação nas conversas, limite e conteúdo das mensagens, formato dos eventos e permissões de administração. Rejeitar tokens inválidos ou expirados. Não confiar em identificadores enviados pelo cliente como prova de autorização.

{ "error": { "code": "CONVERSATION_NOT_FOUND",

"message": "Conversa não encontrada." } }

Utilizar status HTTP adequados: 200, 201, 204, 400, 401, 403, 404, 409 e 500. Erros WebSocket também devem ser estruturados. Recursos privados podem responder 404 em vez de 403 para não revelar sua existência, desde que a decisão seja consistente e documentada.

- Não armazenar nem retornar senhas em texto puro; usar hash apropriado.
- Validar autenticação da conexão WebSocket e autorização em cada operação relevante.
- Isolar conversas, histórico, leitura e notificações de usuários não autorizados.
- Não expor segredos, variáveis sensíveis nem rastros internos em erros.
- Manter .env real fora do Git e fornecer .env.example sem credenciais.

12. Testes e verificação

O participante é responsável por escolher a abordagem de testes e documentar como reproduzir os cenários. Recomenda-se testar cadastro/login; conversas privadas; grupos; mensagens simultâneas;

histórico; autorização; notificações; leitura; múltiplas conexões; reconexão; reenvios; presença; e comunicação via Redis entre duas instâncias.

Testes automatizados são desejáveis, mas ferramentas, abrangência e estratégia são decisões do participante. O projeto deve possuir instruções verificáveis para demonstrar seus fluxos principais.

13. Funcionalidades opcionais

Após atender ao escopo, poderão ser implementados: indicador de digitação, edição ou exclusão lógica de mensagens, busca de histórico, reações, métricas, rate limiting, logs estruturados, Swagger, Docker Compose, CI/CD, testes de carga, Redis Streams e

estratégias de recuperação avançadas. Itens opcionais não substituem requisitos obrigatórios.

14. Autonomia, proatividade e decisões técnicas

Não haverá cronograma de implementação, roteiro de desenvolvimento nem arquitetura previamente definida. Cada participante deverá administrar seu próprio tempo no período de 21 dias, identificar prioridades, pesquisar documentação, comparar soluções, investigar erros e tomar decisões coerentes com o problema.

A ausência de instrução detalhada para uma decisão de implementação não significa que o assunto deva ser ignorado. Espera-se análise do contexto, escolha de uma solução apropriada, reconhecimento de seus limites e registro da decisão.

Registro obrigatório das principais decisões

O README ou documentos complementares devem conter, para cada escolha relevante: (1) decisão tomada; (2) problema ou requisito que a motivou; (3) alternativas consideradas, quando pertinentes;

(4) vantagens e limitações; (5) impactos em manutenção, segurança, desempenho, complexidade ou escalabilidade.

Exemplo de estrutura opcional:

docs/decisions/001-escolha-da-stack.md docs/decisions/002-arquitetura.md docs/decisions/003-modelagem-de-dados.md docs/decisions/004-websocket.md docs/decisions/005-redis.md

Essa estrutura não é obrigatória. A documentação deverá expressar por que a solução foi construída daquela forma, e não apenas o que foi implementado.

15. Documentação obrigatória

- Apresentação do sistema, escopo e funcionalidades efetivamente implementadas.
- Stack utilizada e justificativas técnicas, incluindo alternativas e trade-offs quando relevantes.
- Arquitetura e responsabilidades dos componentes.
- Modelagem, relacionamentos, restrições e decisões sobre consistência.
- Configuração e execução do backend, do banco e do Redis.
- Endpoints HTTP e eventos WebSocket, com exemplos de request/response.
- Fluxo completo de envio, persistência, publicação e entrega de mensagens.
- Estratégias de autenticação, autorização, presença, reconexão e prevenção de duplicatas.
- Instruções de testes, limitações conhecidas e registros de decisões técnicas.

16. Entrega final

Entregar, dentro dos 21 dias, um repositório Git com código-fonte, documentação, arquivos de configuração, scripts/migrations de banco, .env.example e testes quando houver. O histórico de commits deverá permitir acompanhar a evolução do trabalho. Não incluir tokens, senhas ou credenciais reais.

Demonstração funcional esperada

- Dois usuários cadastram-se, autenticam-se e iniciam uma conversa privada.
- Ambos conectam-se via WebSocket e trocam mensagens instantâneas.
- As mensagens aparecem no histórico após desconexão e reconexão.
- Um grupo é criado e recebe mensagens de múltiplos participantes.
- Notificações e marcações de leitura funcionam com autorização.
- Duas instâncias do backend propagam eventos por meio de Redis.
- O participante explica as principais escolhas, problemas encontrados e limites da implementação.

17. Resultado esperado

Uma solução backend funcional, segura, persistente e demonstrável de chat em tempo real, com WebSockets, grupos, conversas privadas, histórico, notificações, presença e Redis. A implementação deve ser tecnicamente coerente e acompanhada de justificativas suficientes para que outra pessoa compreenda como e por que o sistema foi desenvolvido.

O foco é resolver o problema com autonomia e responsabilidade técnica: os requisitos são comuns; as decisões de implementação pertencem aos participantes, sempre justificando suas respostas.